Hough Transforms in Computer Vision

A Complete Module on Parametric Shape Detection

Hough Transforms

Theory • Algorithms • Applications • Case Studies

The Hough Transform is a foundational technique in computer vision for detecting parametric shapes (lines, circles, ellipses, arbitrary objects) in noisy and cluttered images. This module covers the full spectrum: from the classic line detector to advanced variants, real-world applications, and modern optimizations.

Contents

1 Introduction to Hough Transform

1.1 What is the Hough Transform?

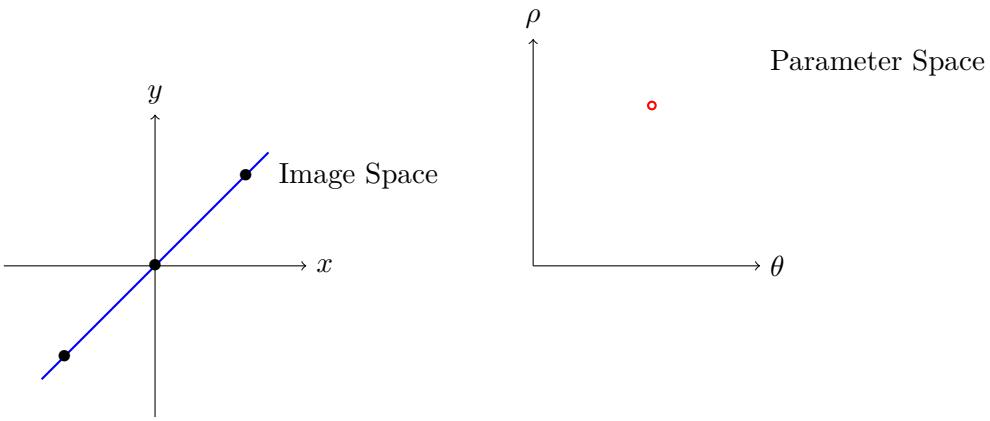
The **Hough Transform (HT)** is a *voting-based* feature extraction method that detects instances of parametric shapes by mapping image points into a parameter space (accumulator). Each point votes for all parameter sets consistent with it, and peaks in the accumulator indicate the presence of a shape.

- **Robust** to noise, gaps, and occlusion.
- **Global** detection — no need for connected components.
- **Parameterizable** — works for any shape with known equation.

1.2 Historical Context

- **1962**: Paul Hough (patent) — line detection via slope-intercept.
- **1972**: Duda & Hart — ρ-θ parameterization (foot-of-normal).
- **1981**: Ballard — Generalized Hough Transform (GHT).
- **1990s–2000s**: Probabilistic, Randomized, Circle, Ellipse variants.

1.3 Core Idea: Voting in Parameter Space



2 Standard Hough Transform (Line Detection)

2.1 Foot-of-Normal Parameterization

A line is represented as:

$$\rho = x\cos\theta + y\sin\theta$$

- ρ: perpendicular distance from origin.
- $\theta \in [0, \pi)$: angle of normal.

Avoids infinite slope problem of $y = mx + c$.

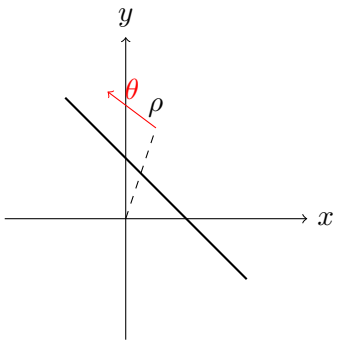


Figure 1: Foot-of-normal representation of a line.

2.2 Algorithm

1. Edge detection → binary edge map $E(x, y)$.
2. Quantize $\rho \in [-\sqrt{2}D, \sqrt{2}D]$, $\theta \in [0, \pi)$.
3. Initialize accumulator $A[\rho][\theta] = 0$.
4. For each edge pixel (x_i, y_i):
$$\rho = x_i \cos\theta + y_i \sin\theta \quad \forall\theta$$
$A[\rho][\theta] + +$
5. Find local maxima → detected lines.

2.3 Example: Line $y = x$

Points: $(1, 1), (2, 2), (3, 3)$
$$\rho = x(\cos\theta + \sin\theta)$$
At $\theta = 45°$:
$$\rho = x\sqrt{2} \quad \Rightarrow \quad \text{Peak at } (\sqrt{2}, 45°)$$

2.4 Line Localization

Recovered line:
$$x\cos\theta + y\sin\theta = \rho$$
Localization error $\propto \Delta\rho, \Delta\theta$.

2.5 Line Fitting vs Hough

	Least Squares	**Hough**
Robustness	Low	High
Outliers	Sensitive	Tolerant
Speed	Fast	Slower

3 Random Sample Consensus (RANSAC) for Line Detection

3.1 Concept

RANSAC (Random Sample Consensus), introduced by Fischler and Bolles in 1981, is a robust model-fitting algorithm that estimates parameters of a mathematical model from a dataset containing *outliers*.

In line detection:

- **Inliers**: Points that lie close to the true line.
- **Outliers**: Noise, background edges, or points from other structures.

RANSAC iteratively samples minimal subsets of data to hypothesize a model, then evaluates how many points support it.

3.2 Why RANSAC for Line Detection?

- Hough Transform uses *all* edge points → sensitive to clutter.
- RANSAC uses *random sampling* → ignores outliers.
- Faster in high-noise scenarios.
- Easily extends to circles, ellipses, homographies.

3.3 Algorithm: RANSAC for Line Detection

1. **Input**: Set of edge points $\mathcal{P} = \{(x_i, y_i)\}$, distance threshold t, iteration count N.
2. **Initialize**: best_inliers $= \emptyset$, best_model = null.
3. **For** $i = 1$ to N:
   (a) Randomly select **2 points** $p_1, p_2 \in \mathcal{P}$.
   (b) Compute line model L_i: slope m, intercept c or (ρ, θ).
   (c) Initialize inlier set $\mathcal{S}_i = \emptyset$.
   (d) **For** each point $p_j \in \mathcal{P}$:
   - Compute perpendicular distance $d_j = |ax_j + by_j + c|/\sqrt{a^2 + b^2}$ (or use ρ-form).
   - If $d_j < t$, add p_j to $\mathcal{S}_i$.

   (e) If $|\mathcal{S}_i| > |$best_inliers$|$:
   - Update best_inliers $= \mathcal{S}_i$
   - Update best_model $= L_i$
4. **Refit** the line using *all* inliers (least squares).
5. **Output**: Final line model and inlier set.

3.4 Key Parameters

Parameter	**Typical Value**
t (distance threshold)	1–3 pixels
N (iterations)	Adaptive or 500–5000
Minimum inliers	$\geq$ 20–50% of points

3.5 Adaptive Iteration Count

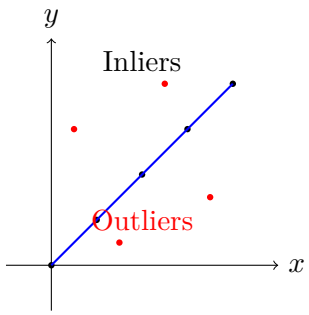
Estimate N from outlier ratio ϵ:

$$N = \frac{\log(1-p)}{\log(1-(1-\epsilon)^s)}$$

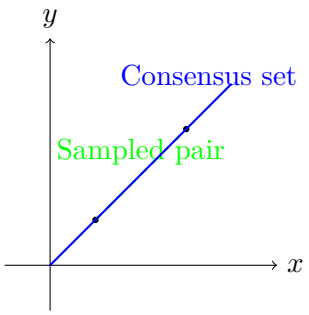
where:

- p = desired probability of success (e.g., 0.99),
- s = sample size (2 for lines),
- ϵ = estimated outlier fraction.

3.6 Example



(a) Data with inliers (black) and outliers (red).



(b) RANSAC samples 2 points → finds consensus.

3.7 RANSAC vs Hough Transform

Aspect	Hough Transform	RANSAC
Mechanism	Voting in parameter space	Random sampling + consensus
Memory	$O(\rho \times \theta)$	$O(1)$
Speed	$O(N \cdot \theta)$	$O(N \cdot k)$
Outlier Handling	Implicit (low votes)	Explicit (ignored)
Multiple Lines	Yes (multiple peaks)	One at a time
Parameter Tuning	Bin size	t, N

Table 1: RANSAC vs Hough for line detection.

3.8 Advantages of RANSAC

- **High robustness** to >50% outliers.
- **L**ow memory usage.
- **F**ast with early termination.
- **E**xtensible to any model (circle, ellipse, homography).

3.9 Disadvantages

- **Probabilistic** — no guarantee of optimal solution.
- **One model at a time** — need sequential RANSAC for multiple lines.
- **Sensitive** to threshold t.

3.10 Sequential RANSAC for Multiple Lines

1. Run RANSAC → find strongest line.
2. Remove its inliers.
3. Repeat until few points remain.